

Project Topics

CSC5201 Advanced Operating Systems

Project-Based Evaluation | Spring 2026

Students are required to undertake one research project from the list below. Each project is designed to support a complete research article suitable for submission to a local journal. Projects emphasize emerging trends in advanced operating systems, distributed systems, and systems software. Deliverables typically include: problem formulation, literature survey, methodology, implementation or simulation (where applicable), analysis, and a written paper in journal format.

Evaluation: Project-based (as per course marks distribution).

Output: A complete draft suitable for a local journal paper (e.g., structure: abstract, introduction, related work, methodology, results, discussion, conclusion, references).

1 Project 1: eBPF-Based Observability and Custom Scheduling in Modern Kernels

Focus

Extended Berkeley Packet Filter (eBPF) enables safe, efficient in-kernel programs for monitoring, tracing, and policy enforcement without kernel module development. This project explores using eBPF for custom observability and scheduling-related instrumentation in Linux.

Research Objectives

- Survey eBPF use cases in production (e.g., Kubernetes, Cilium, Falco) and identify gaps.
- Design and implement (or simulate) an eBPF-based framework for lightweight, application-specific metrics (e.g., per-process cache behavior, I/O patterns).
- Investigate the feasibility of eBPF-assisted scheduling hints or feedback to the kernel scheduler (e.g., co-operation with CFS or real-time schedulers).
- Evaluate overhead, safety, and maintainability compared to traditional kernel modules or userspace agents.

Suggested Methodology

Literature review; design of eBPF programs and (if possible) integration with a recent kernel; benchmarking (latency, throughput, CPU overhead); writing a complete paper with related work, design, implementation, and results.

Emerging Relevance

eBPF is central to modern observability (Prometheus, Grafana, tracing) and security (LSM hooks). Research linking eBPF to OS scheduling and resource management is timely for both industry and academia.

2 Project 2: Lightweight Virtualization for Edge and IoT: Containers vs. MicroVMs

Focus

Edge and IoT deployments need low-footprint, secure isolation. This project compares containers (e.g., Docker, containerd) with lightweight virtual machines (e.g., Firecracker, MicroVM) in terms of performance, security, and resource usage.

Research Objectives

- Systematically compare startup time, memory footprint, and I/O performance of representative container runtimes vs. MicroVMs on the same hardware.
- Analyze security boundaries (kernel attack surface, isolation guarantees) and discuss trade-offs for edge/IoT scenarios.
- Propose a decision framework or hybrid approach for selecting isolation technology based on workload and constraints (latency, memory, multi-tenancy).

- Optionally implement or simulate an edge node manager that can choose between container and MicroVM backends.

Suggested Methodology

Experimental study on a single board or x86 platform; standardized benchmarks (e.g., Redis, Nginx, serverless-style workloads); statistical analysis; threat-model discussion; paper with clear experimental design and reproducible setup.

Emerging Relevance

Edge computing and serverless at the edge rely on fast, cheap isolation. Firecracker-style VMs and containers are both widely used; a rigorous comparison supports practitioners and fills a gap in local/systematic studies.

3 Project 3: Machine Learning for Resource Management and Scheduling in Data Centers

Focus

ML-based approaches for CPU scheduling, memory management, and power management are an active research area. This project surveys and evaluates (or prototypes) ML techniques for OS/data-center resource management.

Research Objectives

- Survey ML for scheduling (e.g., deep reinforcement learning for job scheduling, predictive scaling) and for OS subsystems (memory prefetching, cache partitioning).
- Identify one concrete problem (e.g., batch job scheduling, container placement, or CPU governor tuning) and design an ML-based solution with clear metrics (throughput, latency, power).
- Implement a simulation or integrate with an existing scheduler/simulator (e.g., SimGrid, custom discrete-event simulator) and compare with baseline policies (e.g., FIFO, fair-share).
- Discuss feasibility of deployment in real kernels (e.g., overhead of inference, need for kernel ML frameworks).

Suggested Methodology

Systematic literature review; problem formulation; design of ML model and training pipeline; simulation or limited real-system experiments; ablation studies; paper with related work, methodology, and results.

Emerging Relevance

AI/ML for systems is a major trend; research that bridges ML and classical OS/scheduling is publishable and relevant to cloud and HPC operators.

4 Project 4: Confidential Computing and Trusted Execution Environments in Operating Systems

Focus

Trusted Execution Environments (TEEs) such as Intel SGX, AMD SEV-SNP, and ARM TrustZone provide hardware-assisted confidentiality and integrity. This project investigates how operating systems and middleware can integrate TEEs for secure workloads.

Research Objectives

- Survey TEE architectures, attestation mechanisms, and main limitations (e.g., memory limits, side channels, performance).
- Analyze OS support for TEEs: enclave lifecycle, memory management, I/O and syscall handling (e.g., Gramine, Occlum, LibOS approaches).
- Design or evaluate a use case: e.g., confidential containers, secure key management, or privacy-preserving computation, with focus on OS/kernel integration.
- Discuss threat models, attestation flows, and practical deployment considerations for a specific scenario (e.g., cloud tenant isolation, edge node).

Suggested Methodology

Literature and product survey; optional implementation on a platform with SGX or SEV (or use of emulators/simulators); security and performance evaluation; structured paper with clear threat model and related work.

Emerging Relevance

Confidential computing is critical for multi-tenant clouds and regulated workloads; OS-level research on TEE integration is highly relevant and publishable.

5 Project 5: Persistent Memory and Its Impact on File Systems and Memory Management

Focus

Persistent memory (PMEM), such as Intel Optane or NVDIMM, blurs the line between memory and storage. This project studies how OS design (file systems, memory management, consistency) must evolve for PMEM.

Research Objectives

- Survey PMEM hardware characteristics, programming models (e.g., DAX, libpmem), and existing file systems (e.g., NOVA, ext4-DAX, XFS-DAX).
- Analyze consistency and crash-safety challenges (e.g., ordering, metadata design) and compare design choices in at least two PMEM-aware file systems.
- Design a small prototype or simulation (e.g., a simplified PMEM allocator or a minimal journaling scheme) and evaluate performance and correctness under failure scenarios.

- Discuss implications for virtual memory, page cache, and application programming models.

Suggested Methodology

Literature review; qualitative comparison of file system designs; optional implementation on PMEM-capable hardware or emulation (e.g., QEMU with PMEM); performance and correctness experiments; paper with clear problem statement and design rationale.

Emerging Relevance

PMEM and CXL are changing memory hierarchies; research on OS support is directly applicable to databases, HPC, and large-scale storage systems.

6 Project 6: Operating System Support for Serverless and Function-as-a-Service Platforms

Focus

Serverless (FaaS) platforms impose strict requirements on cold start latency, isolation, and resource accounting. This project investigates OS and runtime mechanisms that enable efficient, secure serverless execution.

Research Objectives

- Survey major FaaS platforms (AWS Lambda, OpenFaaS, Knative) and identify OS/runtime techniques used for fast startup and isolation.
- Analyze trade-offs: snapshotting vs. fresh start, container vs. MicroVM, reuse vs. security.
- Propose or evaluate an OS-level optimization: e.g., lightweight checkpoint/restore, pre-warmed pools, or minimal kernel for serverless (unikernel-style).
- Measure cold start latency, memory footprint, and scalability under a representative workload (e.g., HTTP-triggered functions).

Suggested Methodology

Survey and system comparison; design of an optimization or minimal runtime; implementation or use of existing FaaS frameworks (e.g., OpenFaaS, Apache OpenWhisk) with custom runtime; benchmarking; paper with architecture, experiments, and discussion.

Emerging Relevance

Serverless is a dominant cloud paradigm; OS and systems research that reduces latency and cost is highly relevant to industry and academia.

7 Project 7: Distributed Consensus and Coordination in Modern Cluster Operating Systems

Focus

Cluster OS and orchestration (e.g., Kubernetes) rely on distributed consensus (Raft, Paxos) and coordination (leader election, configuration management). This project studies the role of consensus in OS-level resource and state management.

Research Objectives

- Survey consensus algorithms (Raft, Paxos, Viewstamped Replication) and their use in real systems (etcd, Consul, ZooKeeper) and in OS/orchestration (Kubernetes control plane).
- Analyze consistency guarantees, failure handling, and performance (throughput, latency under partitions) of at least one implementation (e.g., etcd).
- Design or simulate a scenario: e.g., distributed lock service, cluster-wide configuration store, or integration with a custom resource scheduler, and evaluate under faults (node failure, network partition).
- Discuss trade-offs between strong consistency and availability (CAP) in the context of cluster management.

Suggested Methodology

Literature and system review; experimental evaluation of an existing consensus-based system; optional extension or simulation; paper with clear problem, related work, and experimental results.

Emerging Relevance

Distributed systems and cloud-native infrastructure depend on consensus; research that connects classical distributed systems theory to modern cluster OS is publishable and educational.

8 Project 8: Energy-Aware and Green Computing in Operating Systems

Focus

Energy efficiency is critical in data centers and edge devices. This project investigates OS-level mechanisms for power and energy management: scheduling, frequency scaling, and workload placement.

Research Objectives

- Survey OS and runtime support for power management: CPU governors, idle states, heterogeneous cores (big.LITTLE), and GPU power management.
- Identify metrics (e.g., energy-delay product, joules per task) and methodologies for measuring energy at the OS or process level.

- Design or evaluate a policy: e.g., energy-aware scheduler, batch consolidation to allow deeper sleep, or placement of tasks on efficiency vs. performance cores.
- Conduct experiments on real hardware (e.g., ARM or x86 with RAPL) or simulation, and compare with default OS behavior.

Suggested Methodology

Literature review; selection of platform and measurement tools (e.g., perf, RAPL, external power meter); design of energy-aware policy; experimental evaluation; paper with methodology, results, and discussion of applicability.

Emerging Relevance

Sustainability and cost drive interest in green computing; OS-level research on energy management is relevant to both embedded and data-center domains.

9 Project 9: Security Hardening and Attack Mitigation in the Linux Kernel

Focus

Kernel security is under constant pressure from new attack vectors (e.g., side channels, use-after-free, privilege escalation). This project surveys and evaluates hardening mechanisms and mitigations in the Linux kernel.

Research Objectives

- Survey kernel hardening features: KASLR, stack canaries, Control Flow Integrity (CFI), memory safety (e.g., sanitizers), and LSM frameworks (SELinux, AppArmor).
- Analyze one family of attacks (e.g., cache side channels, Spectre/Meltdown variants) and the corresponding kernel mitigations, including performance impact.
- Evaluate (or simulate) the effectiveness and cost of a subset of mitigations on a recent kernel (e.g., 5.x or 6.x) using benchmarks and optional attack PoC.
- Propose or discuss a hardening strategy for a specific scenario (e.g., multi-tenant container host, IoT gateway).

Suggested Methodology

Literature and kernel documentation review; systematic comparison of mitigations; experimental measurement of performance and (if feasible) security; paper with threat model, related work, and recommendations.

Emerging Relevance

Kernel security is a permanent concern; research that systematizes mitigations and their trade-offs supports practitioners and aligns with emerging trends in secure systems.

10 Project 10: Unikernels and Library Operating Systems for Specialized Workloads

Focus

Unikernels and library OS (e.g., IncludeOS, OSv, Unikraft) compile the application with a minimal OS layer into a single image, reducing footprint and attack surface. This project evaluates their applicability to specific workloads (e.g., network functions, serverless).

Research Objectives

- Survey unikernel and library OS projects: design principles, supported languages and runtimes, and target platforms (bare metal, VM, container).
- Compare boot time, memory footprint, and throughput with a general-purpose OS (e.g., Linux) for at least one workload (e.g., key-value store, HTTP server, packet processing).
- Analyze limitations: compatibility (syscalls, drivers), debugging, and ecosystem support.
- Propose a use case where unikernels offer clear benefits (e.g., edge node, FaaS runtime) and discuss migration path and trade-offs.

Suggested Methodology

Literature and project survey; deployment of one or more unikernels (e.g., Unikraft, OSv) and baseline Linux; benchmarking and qualitative comparison; paper with design overview, experiments, and discussion of applicability.

Emerging Relevance

Unikernels remain relevant for niche, performance- and security-critical workloads; research that clarifies their strengths and limitations supports adoption and further development.

General Guidelines for the Research Paper

- **Structure:** Abstract, Introduction (motivation, objectives, contributions), Related Work, Methodology/Design, Implementation or Experimental Setup, Results and Analysis, Discussion, Conclusion, References.
- **Length:** Aim for a length suitable for a local journal (e.g., 5000–8000 words or as per target journal guidelines).
- **Ethics:** Any security testing or attack PoC must be conducted in controlled environments and comply with institutional and legal guidelines.
- **Reproducibility:** Where possible, provide enough detail (hardware, software versions, parameters) for results to be reproduced.